

INFORME TÉCNICO: TRABAJO PRÁCTICO INTEGRADOR

Diseño de la Capa de Datos para un Sistema RAG de Documentación Técnica

Asignatura: Bases de Datos para Inteligencia Artificial

Docente: Martín Lacheski

Año: 2026

Integrantes del Grupo: Andrea Ferenaz, Mauro Virgilio Blanc, Juan Pablo Imbrogno

1. Descripción del Caso de Uso

1.1 Contexto y Problema a Resolver

La organización cuenta con un volumen considerable de documentación técnica — manuales de procedimiento, especificaciones de maquinaria, normativas de seguridad e instructivos operativos— distribuida de forma heterogénea en archivos PDF e históricos. El proceso actual de búsqueda de información es manual, ineficiente y dependiente del conocimiento tácito de los empleados con mayor antigüedad, generando cuellos de botella en la resolución de incidentes operativos.

Para resolver este problema, se diseña la infraestructura de datos para un sistema de **Generación Aumentada por Recuperación (RAG - *Retrieval-Augmented Generation*)**. La solución permite ingestar la documentación, procesarla en fragmentos consultables (*chunks*), vectorizarlos y permitir que los usuarios realicen consultas en lenguaje natural mediante un Modelo de Lenguaje (LLM). El sistema recupera los fragmentos con mayor similitud semántica y los entrega como contexto al LLM, garantizando respuestas precisas y citando las fuentes utilizadas.

1.2 Usuarios y Roles Principales

- **Técnico / Ingeniero (Lector):** Consulta dudas operativas en lenguaje natural. Solo debe acceder a la información autorizada para su área y nivel jerárquico.
- **Administrador de Documentación (Editor):** Carga nuevos documentos, actualiza versiones, define clasificaciones y retira documentación obsoleta.
- **Auditor / Supervisor (Auditor):** Revisa el historial de consultas, evalúa las fuentes recuperadas y analiza métricas de precisión y trazabilidad de las respuestas generadas por la IA.

1.3 Decisiones de Diseño Críticas y Gestión de Riesgos

1. **Aislamiento y Filtrado de Seguridad (Fuga de Información):** El mayor riesgo en sistemas RAG corporativos es el retorno no autorizado de fragmentos

confidenciales. El diseño aborda esto integrando metadatos de seguridad directamente en las consultas vectoriales y aplicando políticas a nivel de base de datos relacional (*Row Level Security*).

2. **Trazabilidad y Auditoría:** La base de datos registra explícitamente qué versión de qué documento y qué fragmentos específicos se utilizaron para responder cada consulta.
3. **Control de Obsolescencia:** Un documento marcado como "archivado" o "obsoleto" inhabilita automáticamente sus fragmentos vectoriales en el pipeline de búsqueda RAG.

2. Relevamiento y Clasificación de Datos

Para alimentar el pipeline RAG y la capa operacional de soporte, los datos se clasifican según su naturaleza y nivel de estructuración:

Tipo de Dato	Descripción y Ejemplos	Formato / Almacenamiento
No Estructurados	Archivos PDF originales, manuales en Markdown y diagramas embebidos.	Almacenamiento de Objetos (S3 / Blob Storage).
Semiestructurados	Fragmentos de texto (<i>chunks</i>) extraídos, metadatos de párrafo, palabras clave y respuestas generadas en formato JSON.	Tipos <code>JSONB</code> en PostgreSQL / Colecciones NoSQL.
Estructurados (Operacionales)	Entidades del dominio: Usuarios, Roles, Áreas, Categorías, Documentos y Versiones.	Tablas Relacionales (PostgreSQL).
Estructurados (Vectoriales)	Representaciones matemáticas (<i>embeddings</i>) de 1536 dimensiones de cada fragmento de texto.	Tipo <code>vector(1536)</code> mediante la extensión <code>pgvector</code> .

Tipo de Dato	Descripción y Ejemplos	Formato / Almacenamiento
Datos de Auditoría	Logs de consultas realizadas por usuarios, tiempo de respuesta, prompt emitido y relación M:N de fragmentos citados.	Tablas relacionales con restricciones de clave foránea.

3. Modelo Conceptual

El dominio se modeló mediante un Diagrama Entidad-Relación (DER) que independiza la lógica del negocio de la implementación tecnológica final.

3.1 Entidades y Atributos Principales

- **AREA:** area_id (PK), nombre_area, codigo_interno.
- **ROL:** rol_id (PK), nombre_rol, nivel_jerarquico.
- **USUARIO:** usuario_id (PK), nombre, email, area_id (FK), rol_id (FK).
- **CATEGORIA:** categoria_id (PK), nombre_categoria, descripcion.
- **DOCUMENTO:** documento_id (PK), titulo, estado (Vigente/En Revisión/Archivado), nivel_acceso_requerido, area_id (FK), categoria_id (FK).
- **VERSION_DOCUMENTO:** version_id (PK), documento_id (FK), numero_version, fecha_publicacion, url_archivo_origen, es_version_activa (Boolean).
- **FRAGMENTO (Chunk):** fragmento_id (PK), version_id (FK), numero_secuencia, contenido_texto, embedding_vector (Vector), metadatos_json (JSONB).
- **CONSULTA_RAG:** consulta_id (PK), usuario_id (FK), pregunta_texto, respuesta_ia, fecha_hora.
- **FUENTE_CONSULTA (Relación M:N):** consulta_id (FK), fragmento_id (FK), score_similitud.

4. Modelo Lógico y Selección Tecnológica

4.1 Justificación de la Selección Tecnológica

Se seleccionó **PostgreSQL con la extensión pgvector** como motor unificado de la solución.

Razones de la elección:

1. **Unificación de Arquitectura (Hybrid Search):** Evita la necesidad de mantener dos bases de datos separadas (una relacional para usuarios/permisos y una vectorial dedicada como Pinecone/Qdrant).
2. **Consistencia ACID y Filtrado Unificado:** Permite ejecutar la búsqueda de similitud semántica por coseno y el filtrado estricto de seguridad relacional (`WHERE area_id = ...`) en una sola consulta SQL atómica.
3. **Soporte de Metadatos Semiestructurados:** El tipo de dato nativo `JSONB` de PostgreSQL permite almacenar y filtrar metadatos cambiantes de cada fragmento (página, párrafo, tags) con soporte de índices GIN.

5. Decisiones de Normalización y Diseño

- **Normalización Relacional (Hasta 3FN):** La capa operacional (`USUARIO`, `DOCUMENTO`, `VERSION_DOCUMENTO`, `AREA`) está estrictamente normalizada en Tercera Forma Normal para garantizar la integridad referencial, eliminar redundancias y prevenir anomalías de actualización en datos críticos de seguridad.
- **Desnormalización Controlada en Fragmentos:** En la tabla `FRAGMENTO`, se duplica de manera calculada el campo de metadatos en un objeto `JSONB`. Esto optimiza la velocidad de lectura durante el pipeline de recuperación del LLM sin requerir *joins* complejos en el momento de la consulta vectorial de baja latencia.

6. Modelo Físico e Implementación Mínima

El esquema físico se implementó mediante DDL en PostgreSQL, habilitando la extensión `pgvector`.

SQL

```
-- Habilitación de la extensión vectorial
CREATE EXTENSION IF NOT EXISTS vector;

-- Tabla de Áreas
CREATE TABLE areas (
    area_id SERIAL PRIMARY KEY,
    nombre_area VARCHAR(100) NOT NULL,
    codigo_interno VARCHAR(20) UNIQUE NOT NULL
);

-- Tabla de Usuarios
CREATE TABLE usuarios (
    usuario_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    area_id INT NOT NULL REFERENCES areas(area_id),
    nivel_acceso VARCHAR(50) NOT NULL DEFAULT 'Tecnico_Basico'
);

-- Tabla de Documentos
```

```

CREATE TABLE documentos (
    documento_id SERIAL PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    estado VARCHAR(20) CHECK (estado IN ('vigente', 'en_revision',
'archivado')) DEFAULT 'vigente',
    nivel_acceso_requerido VARCHAR(50) NOT NULL,
    area_id INT NOT NULL REFERENCES areas(area_id)
);

-- Tabla de Versiones
CREATE TABLE versiones_documento (
    version_id SERIAL PRIMARY KEY,
    documento_id INT NOT NULL REFERENCES documentos(documento_id),
    numero_version VARCHAR(20) NOT NULL,
    fecha_publicacion TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    url_archivo_origen VARCHAR(500) NOT NULL,
    es_activa BOOLEAN DEFAULT true
);

-- Tabla de Fragmentos (Chunks) con soporte Vectorial
CREATE TABLE fragmentos (
    fragmento_id SERIAL PRIMARY KEY,
    version_id INT NOT NULL REFERENCES versiones_documento(version_id),
    numero_secuencia INT NOT NULL,
    contenido_texto TEXT NOT NULL,
    embedding vector(1536), -- Dimensión para OpenAI text-embedding-3-
small / Ada-002
    metadatos JSONB
);

-- Tabla de Consultas RAG
CREATE TABLE consultas_rag (
    consulta_id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES usuarios(usuario_id),
    pregunta_texto TEXT NOT NULL,
    respuesta_ia TEXT NOT NULL,
    fecha_hora TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Tabla Intermedia para Fuentes (Trazabilidad M:N)
CREATE TABLE fuentes_consulta (
    consulta_id INT REFERENCES consultas_rag(consulta_id) ON DELETE
CASCADE,
    fragmento_id INT REFERENCES fragmentos(fragmento_id),
    score_similitud FLOAT NOT NULL,
    PRIMARY KEY (consulta_id, fragmento_id)
);

```

7. Consultas Representativas

A continuación se presentan las consultas SQL clave que resuelven la lógica de negocio, optimización y trazabilidad del sistema.

Consulta 1: Búsqueda RAG por Similitud Coseno con Filtro de Seguridad Integrado

- **Pregunta de negocio:** *"Buscar los 3 fragmentos de texto más parecidos a la pregunta del usuario '¿Cómo actuar ante una vibración excesiva en la turbina?', garantizando que el usuario solo acceda a documentos de su área y con estado vigente."*

SQL

```
SELECT
    f.fragmento_id,
    d.titulo AS documento_titulo,
    f.contenido_texto,
    f.metadatos->>'pagina_pdf' AS pagina,
    1 - (f.embedding <=> '[0.0142, -0.0831, 0.3124, ...]':vector) AS
similitud_coseno
FROM fragmentos f
JOIN versiones_documento v ON f.version_id = v.version_id
JOIN documentos d ON v.documento_id = d.documento_id
JOIN usuarios u ON u.usuario_id = 10 -- ID del usuario que consulta
WHERE v.es_activa = TRUE
    AND d.estado = 'vigente'
    AND d.area_id = u.area_id -- Filtro de Aislamiento
ORDER BY f.embedding <=> '[0.0142, -0.0831, 0.3124, ...]':vector ASC
LIMIT 3;
```

Consulta 2: Auditoría y Trazabilidad de Fuentes por Consulta

- **Pregunta de negocio:** *"Dado el ID de una consulta generada por la IA, determinar qué documentos, versiones y fragmentos exactos fueron utilizados para construir la respuesta."*

SQL

```
SELECT
    c.consulta_id,
    c.pregunta_texto,
    c.respuesta_ia,
    d.titulo AS documento_fuente,
    v.numero_version,
    f.numero_secuencia AS chunk_numero,
    fc.score_similitud
FROM consultas_rag c
JOIN fuentes_consulta fc ON c.consulta_id = fc.consulta_id
JOIN fragmentos f ON fc.fragmento_id = f.fragmento_id
JOIN versiones_documento v ON f.version_id = v.version_id
JOIN documentos d ON v.documento_id = d.documento_id
WHERE c.consulta_id = 5001
ORDER BY fc.score_similitud DESC;
```

Consulta 3: Reporte Analítico de Documentos más Citados por la IA

- **Pregunta de negocio:** *"¿Cuáles son los 5 documentos técnicos más consultados y citados por el sistema RAG en el último mes?"*

SQL

```
SELECT
    d.documento_id,
    d.titulo,
    COUNT(fc.fragmento_id) AS total_citas
FROM fuentes_consulta fc
JOIN fragmentos f ON fc.fragmento_id = f.fragmento_id
JOIN versiones_documento v ON f.version_id = v.version_id
JOIN documentos d ON v.documento_id = d.documento_id
JOIN consultas_rag c ON fc.consulta_id = c.consulta_id
WHERE c.fecha_hora >= NOW() - INTERVAL '30 days'
GROUP BY d.documento_id, d.titulo
ORDER BY total_citas DESC
LIMIT 5;
```

Consulta 4: Detección de Documentos Obsoletos con Fragmentos Activos (Control de Integridad)

- **Pregunta de negocio:** *"Listar las versiones de documentos marcadas como inactivas o archivadas que aún poseen fragmentos indexados, para mantenimiento del pipeline."*

SQL

```
SELECT
    d.titulo,
    v.numero_version,
    d.estado,
    COUNT(f.fragmento_id) AS cantidad_chunks_asociados
FROM documentos d
JOIN versiones_documento v ON d.documento_id = v.documento_id
JOIN fragmentos f ON v.version_id = f.version_id
WHERE d.estado = 'archivado' OR v.es_activa = FALSE
GROUP BY d.titulo, v.numero_version, d.estado;
```

8. Estrategia de Optimización e Índices

Para garantizar tiempos de respuesta en milisegundos en búsquedas vectoriales sobre miles de fragmentos, se implementa un índice vectorial **HNSW (Hierarchical Navigable Small World)** usando la distancia de coseno:

SQL

```
-- Índice vectorial HNSW para distancia coseno
CREATE INDEX idx_fragmentos_embedding_hnsw
ON fragmentos
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

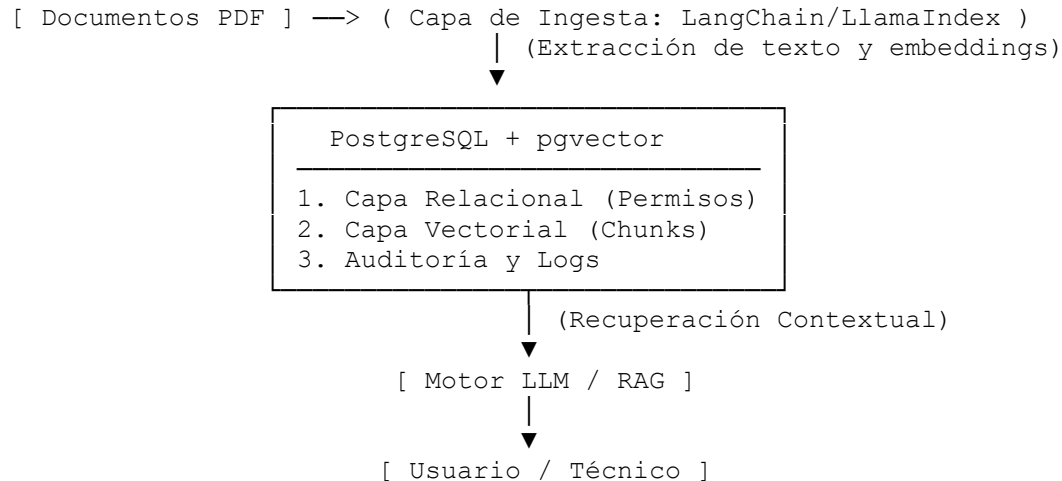
-- Índice GIN sobre metadatos JSONB
CREATE INDEX idx_fragmentos_metadatos_gin
ON fragmentos
USING gin (metadatos);
```

```
-- Índice B-Tree para optimizar los Joins relacionales
CREATE INDEX idx_documentos_area_estado
ON documentos (area_id, estado);
```

9. Arquitectura de Datos

El flujo de información se organiza en 4 capas secuenciales:

Plaintext



10. Seguridad, Permisos y Escalabilidad

- Seguridad y Aislamiento (Multi-tenant por Área):** La seguridad se aplica mediante filtrado obligatorio en la cláusula `WHERE` del motor SQL combinando `area_id` y `nivel_acceso_requerido`. Adicionalmente, se propone la activación de **Row Level Security (RLS)** en PostgreSQL para limitar las filas retornadas según el rol de la sesión de base de datos del usuario autenticado.
- Escalabilidad:**
 - Particionado:** La tabla de fragmentos se puede particionar por rango según la categoría o área del documento.
 - Mantenimiento de Índices:** Monitoreo periódico del parámetro `ef_search` del índice HNSW para balancear rendimiento de CPU vs. precisión (*recall*) de la búsqueda semántica.

11. Conclusiones

El diseño desarrollado demuestra que una solución de datos para Inteligencia Artificial no requiere necesariamente desacoplar los motores de búsqueda de la base de datos operacional. La adopción de **PostgreSQL con pgvector** permite resolver el problema de recuperación semántica propio de la arquitectura RAG manteniendo estrictos controles de integridad relacional, seguridad por roles y trazabilidad completa de las fuentes de información utilizadas por el LLM.